

영상 기반 경량 World-Action Model

대상 환경은 레일 위 카트와 그 위에 매달린 진자 2개입니다. 모델은 화면만으로 카트 속도와 진자 각도, 관측되지 않는 질량·마찰·모터 반응까지 추정해야 합니다. 참가자는 이 추정으로 영상 예측과 행동 선택을 함께 수행하는 경량 World-Action Model을 제출합니다.

미래 예측

행동 이후의 장면을 예측하는 유효 길이

행동 선택

같은 상태 표현으로 진자 2개를 세우고 유지하는 제어 성능

문제 개요: World-Action Model

문제 환경은 레일 위 카트와 그 위에 매달린 이중진자입니다. 정책은 행동을 선택하고 시뮬레이터는 행동의 결과를 계산합니다. World-Action Model은 이 두 기능을 하나의 학습된 표현으로 수행하며, 환경의 변화를 예측하고 같은 표현으로 다음 행동을 선택합니다.

월드 모델(world model)은 화면과 행동 기록에서 환경의 변화 규칙을 학습해, 실제 환경을 실행하지 않고도 행동 이후의 화면을 예측합니다. 여기에 같은 표현으로 다음 행동을 선택하는 기능을 더한 것이 World-Action Model(WAM)입니다.

예측은 물리적으로 정확해야 합니다. 카트를 밀었을 때 진자가 움직이는 방향과 크기, 질량과 마찰에 따른 반응 차이를 구분해야 합니다. 틀린 예측 위에서 선택한 행동은 오래 유지되지 않습니다.

소비자용 GPU 1장으로 학습하고 실시간으로 실행할 수 있는 소형 모델 안에 상태 추정, 동역학 예측, 행동 선택 기능을 얼마나 효율적으로 구현하는지를 평가합니다. 학습에 쓸 수 있는 자료의 범위는 아래 학습 자료 절에 있습니다.

핵심 과제

환경과의 온라인 상호작용 없이, 배포된 오프라인 데이터로 미래 프레임 예측과 행동 선택을 함께 수행하는 경량 World-Action Model 학습

문제 환경: 카트와 이중진자

카트는 레일을 따라 좌우로 움직이며 그 위에 관절로 연결된 진자 2개가 매달려 있습니다. 제어 명령은 카트를 미는 방향과 힘의 크기 하나뿐입니다.

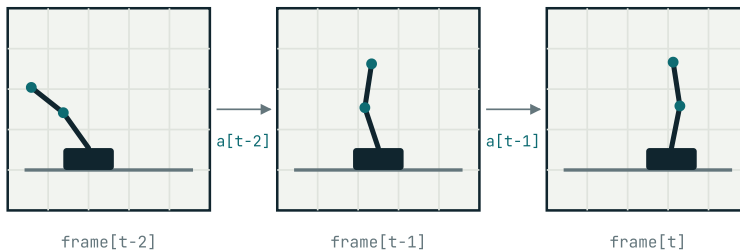
카트의 위치·속도와 진자의 각도·각속도는 수치로 제공하지 않습니다. 모델은 연속된 화면과 그 사이의 액션에서 현재 상태를 추정해야 합니다.

관측 · $o[t]$

25 Hz로 입력되는 $128 \times 128 \times 3$ RGB 영상입니다. 수치 상태, segmentation mask, depth map은 제공하지 않습니다.

액션 · $a[t]$

$[-1, 1]$ 범위의 실수 1개입니다. 음수는 왼쪽, 양수는 오른쪽이며 절댓값이 힘의 크기입니다. 0 은 힘을 가하지 않는 명령입니다.



화면 스타일과 두 진자의 길이는 에피소드마다 다르며 한 에피소드 안에서는 고정됩니다.

관측: 화면만 제공

$128 \times 128 \times 3$ RGB · 25 Hz

카트 위치·진자 각도 등 상태 벡터는 제공하지 않음 · 픽셀에서 직접 추정

액션: 실수 1개

$a \in [-1, 1]$ · 카트를 미는 좌우 힘



- 왼쪽 · + 오른쪽

그림 1 · Micro WAM의 관측과 액션. 입력은 카트와 이중진자가 담긴 128×128 RGB 화면과 자신이 출력한 액션뿐이며, 각도·각속도 등 상태 값은 화면에서 추정합니다.

에피소드마다 달라지는 것은 두 진자의 길이입니다. 공개 범위 안에서 매 에피소드 독립적으로 뽑히며, 링크의 선밀도가 고정이므로 질량과 관성은 길이에서 따라옵니다. 모델은 짧은 움직임에서 이번 에피소드의 길이와 그에 따른 반응 특성을 파악해야 합니다.

반면 모터 게인, 링크 선밀도, 그리고 표준 이중진자 방정식으로 설명되지 않는 매끄러운 추가 반응은 모든 에피소드에서 동일하게 고정됩니다. 추가 반응의 구체적인 구성은 공개하지 않으나, 배포 데이터가 같은

환경에서 수집되므로 데이터 안에 온전히 담겨 있습니다. 즉 이 부분은 데이터로부터 한 번 학습해 두면 되는 대상입니다.

장면은 **단색 방**입니다. 뒤쪽 벽과 바닥, 그 경계, 방향성 조명 하나와 드리운 그림자로 이루어집니다.

물체의 색은 모든 에피소드에서 동일하게 고정됩니다 — 진자 두 개는 같은 색이고, 레일 · 카트 · 관절 · 벽 · 바닥이 각각 다른 고정색입니다. 에피소드마다 달라지는 것은 **카메라와 조명, 그리고 장비의 전체 크기**입니다: 화면 확대 비율, 카메라 높이, 바닥까지의 거리, 조명 방향과 세기, 대비, 비네팅, 그림자의 오프셋과 진하기, 센서 그레인, 렌즈 초점 흐림, 그리고 장비 자체의 크기. 한 에피소드 안에서는 모두 고정입니다. 모델은 외형의 변화와 운동 법칙의 변화를 구분해야 합니다.

물리든 외형이든 모든 축은 공개된 범위 안에서 뽑힙니다. 채점에도 같은 축과 범위만 씁니다. 축은 전부 연속값이며 에피소드마다 독립적으로 뽑힙니다. 외형 축의 전체 목록과 각 범위는 스타터 킷에 공개합니다.

화면 안에는 크기가 고정된 기준물이 없습니다. 진자 길이를 제외한 모든 렌더 치수 — 장비 전체 크기, 레일이 이동 한계를 넘어 뺀 여유, 바닥까지의 거리, 그림자 오프셋 — 는 물리와 무관하게 에피소드마다 달라지며, 그 변동 폭은 화면 확대 비율의 변동 폭보다 넓습니다.

정지 화면 한 장도 길이에 대한 정보를 일부 담고 있습니다. 여러 치수를 동시에 관측하면 환산 계수의 범위가 좁혀지며, 그 정보량은 주최 측이 측정해 스타터 킷에 공개합니다. 실용적으로 필요한 정밀도는 움직임에서 나옵니다 — 진동 주기가 길이의 제공근에 비례합니다.

레일은 **카트의 이동 한계 바로 바깥에서 끝나며 양 끝에 바닥까지 닿는 다리가 있습니다.** 카트가 움직일 수 있는 범위는 이렇게 화면에 표시됩니다. 레일이 한계를 얼마나 넘어 뺀지는 에피소드마다 달라지므로, 다리의 위치는 정지 장치의 위치를 근사적으로 나타냅니다. 레일 양 끝에는 정지 장치가 있습니다. 정지 장치에 닿은 카트는 그 자리에서 멈추고(비탄성), 그 충격은 진자에 전달됩니다.

평가 구간 물리 궤적의 결정성

동일한 비공개 초기 상태와 동일한 액션열은 동일한 결과를 냅니다. 러너가 공급하는 초기 잠재값의 무작위성은 환경의 물리 외란과 별개입니다.

제공 자료와 학습 범위

학습의 중심은 주최 측이 여러 물리 조건과 화면 스타일에서 수집한 오프라인 데이터이며, 그 밖에 쓸 수 있는 자료의 범위는 아래에 있습니다.

전체 패키지는 25 GB 이하이고, 첫날부터 작업할 수 있도록 10% 규모의 개발용 부분집합(2 GB 이하)을 함께 배포합니다. 프레임 수는 스타터 킷의 데이터 명세에 명시합니다. 각 시퀀스는 한 에피소드 (400~1000프레임, 16~40초) 안의 25 Hz 영상이며 길이를 표기합니다. **에피소드의 약 10 %는 영상과 그 사이의 액션**을 같은 시간축으로 제공하고, **나머지는 RGB 영상만** 제공합니다. 에피소드 경계를 명시하며, 한 시퀀스는 하나의 물리 조건입니다. 영상은 **무손실**로 저장되어 평가에 쓰는 렌더와 바이트 단위로 같으며, 한 번에 풀어 쓰는 memmap 변환 스크립트를 함께 제공합니다.

영상만 있는 시퀀스로는 다양한 외형에 공통된 운동을, 액션이 있는 시퀀스로는 카트를 민 방향과 이후 반응의 관계를 학습할 수 있습니다. 보상 값은 배포하지 않습니다. 제어 점수의 정의(3-2)는 공개되므로, 제어 목표에 유리한 상태가 무엇인지는 모델이 학습한 상태 표현 안에서 스스로 판단해야 합니다. 학습 단계에서 참가자가 프레임으로부터 상태나 보상을 스스로 추정해 라벨로 쓸 수 있습니다. 아래의 제한은 제출 그래프에만 적용됩니다.

액션 데이터에는 약한 움직임 외에 진자를 크게 흔들어 에너지를 높이는 구간, 직립 상태에서 시작해 균형을 유지하거나 잃는 구간, 밀기를 받아 버티거나 넘어진 뒤 다시 세우는 구간, 실패로 이어지는 움직임이 포함됩니다. 밀기는 3-2 평가와 같은 방식이며 그 시점과 크기는 제공하지 않습니다. 매달린 상태에서 흔들어 올려 오래 세우는 전체 주행을 한 에피소드에 담은 시범은 제공하지 않습니다. 모델은 기록을 복사하는 대신 학습한 월드 모델 안의 행동 결과를 활용해야 합니다.

스타터 킷에는 공통 시각 stem의 구조와 참조 가중치, 제출 템플릿(그래프별 함수 형식과 비워 둔 TODO 블록), ONNX 러너, 그래프 검증기가 포함됩니다. 채점 지표는 이 문제문의 정의가 기준입니다.

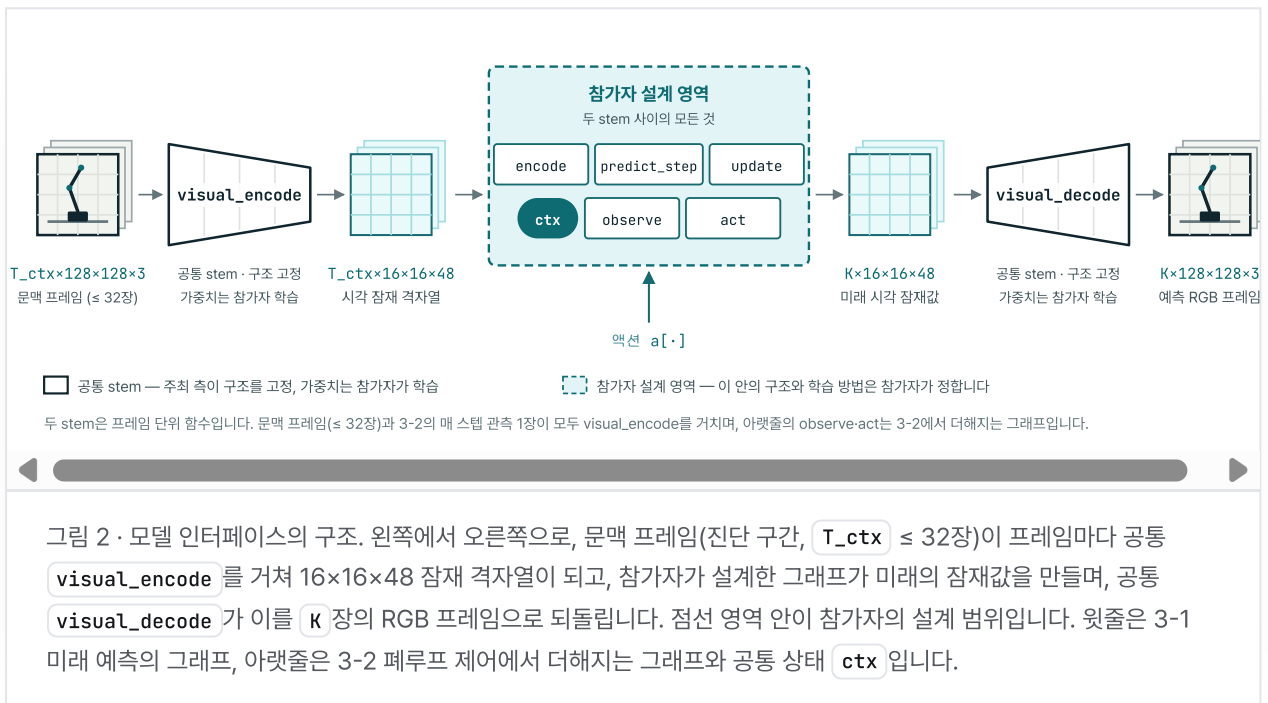
학습 데이터의 범위

공개적으로 이용 가능한 외부 데이터셋과 사전학습 가중치는 **사용할 수 있습니다**. 사용한 것은 매니페스트와 보고서에 선언하며, 가져온 가중치도 파라미터 수에 함께 집계합니다. 학습에 쓸 수 있는 것은 배포 데이터와 위 외부 자료이며, 환경과의 온라인 상호작용은 불가능합니다. 그 밖의 학습 방법은 자유입니다.

모델 인터페이스: 공통 시각 stem과 자유 예측기

주최 측이 고정하는 것은 **시각 경계**입니다. RGB 화면을 잠재 격자로 바꾸는 공통 encoder와 잠재 격자를 RGB로 되돌리는 공통 decoder의 구조를 스타터 킷으로 제공하며, 그 **가중치**는 참가자가 배포 데이터로 학습합니다(스타터 킷의 참조 가중치를 그대로 쓰거나 재학습할 수 있습니다). 두 stem 사이의 모든 것은 참가자가 설계합니다.

생성 알고리즘은 자유입니다. 필수 조건은 세 가지입니다: 과거 영상과 행동으로부터 현재의 예측 상태 **ctx** 를 만들 것, 미래 행동에 조건화하여 다음 시각 잠재값을 예측할 것, 자기 예측을 다시 입력으로 사용해 정답 관측 없이 미래를 계속 이어갈 것.



미래 예측은 하나의 함수로 통합됩니다. **predict_step(z, level, step, actions[K], ctx)** 는 **갱신된 잠재값**을 반환합니다. 참가자는 호출 횟수 N 을 선언하며, $N = 1$ 이면 한 번에 예측하는 direct 모델, $N > 1$ 이면 반복 정제 모델입니다. 두 방식은 N 으로만 구분됩니다. 각 블록의 시작 잠재값은 러너가 공표된 시드 묶음에서 **runner.initial_latent(seed)** 로 공급하고, **level** 과 **step** 은 정제 진행 정보입니다. 결정론적 모델은 이 셋을 무시할 수 있습니다.

참가자는 매니페스트에 **생성 단위** K (한 블록에서 함께 예측하는 프레임 수 — 1이면 프레임 단위 자기회귀, 2 이상이면 청크 단위)와 **예측 호출 수** N 을 선언합니다. $K \leq 8$, $N \leq 32$ 이며, 여기에 **프레임당 호출 수 제한** $N \leq 4K$ 가 함께 적용됩니다 — $K = 1$ 이면 $N \leq 4$ 이고, $N = 32$ 는 $K = 8$ 에서만 선언할 수 있습니다. 이 규칙으로 프레임당 **predict_step** 호출 수의 상한은 K 와 무관하게 같습니다.

K = 1 · 프레임 단위 자기회귀



K = 4 · 청크 단위 · 여러 프레임을 한 블록으로 함께 예측



어느 쪽이든 predict_step 호출은 선언된 N회,
블록이 끝날 때마다 update로 ctx를 갱신합니다.

생성 시간 기준: 프레임당 시간 = 블록 시간 ÷ K

점선 원은 러너가 공급하는 초기 잠재값, 반투명 블록은 다음 반복입니다.

그림 3 · 생성 단위 **K**의 선언. K=1이면 프레임 단위 자기회귀, K>1이면 여러 프레임을 한 블록으로 함께 예측하는 청크 생성입니다. 어느 쪽이든 **predict_step**은 선언된 횟수 **N**회 호출되며, 블록이 끝날 때마다 **update**가 **ctx**를 갱신합니다.

함수	역할	호출 시점
visual_encode(frame) 공통 stem	RGB 화면 1장을 16×16×48 시각 잠재 격자로 변환. 원본 RGB는 이 함수에만 전달됩니다	진단 구간의 매 프레임 · 3-2의 매 스텝
encode(grid[T_ctx], actions[T_ctx-1])	진단 구간의 잠재 격자열과 액션을 공통 상태 표현 ctx 로 압축	두 소문제 시작 시
observe(ctx, grid_next, action_prev)	환경에서 새로 받은 화면의 잠재 격자로 ctx 를 갱신	3-2 페루프 제어
predict_step(z, level, step, actions[K], ctx)	블록 잠재값을 한 단계 갱신해 반환. 블록마다 N 회 호출	3-1 미래 예측
visual_decode(z_final) 공통 stem	완성된 블록 잠재값 z_final 만 받아 K 개의 RGB 프레임으로 변환	3-1 영상 출력
update(ctx, z_final, actions[K])	자기가 예측한 잠재값과 액션을 ctx 에 반영. 정답 프레임은 제공하지 않습니다	3-1 개루프 롤아웃
act(ctx)	공통 상태 표현에서 다음 좌우 힘 1개를 출력	3-2 행동 선택

공통 stem의 범위. encoder stem은 합성곱 특징 추출까지만 담당하고, 그 이후는 모두 참가자의 설계입니다. decoder stem은 **z_final**만 받으므로 배경·카메라·조명 등 장면 재현에 필요한 정보도

모두 잠재값 안에 담겨야 합니다. stem의 가중치는 참가자가 학습해 제출하므로 파라미터 수에 포함되며, 구조가 고정이라 그 몫은 모두에게 같은 상수입니다 — 스타터 킷에 수치로 공표합니다.

원본 RGB는 `visual_encode`에만 전달됩니다. 환경에 특화된 비학습식 영상 분석 경로는 허용하지 않습니다.

학습 조건. 제출 모델은 실제 미래 관측 또는 그 표현을 목표로 하는 **행동 조건부 미래 예측 목적함수**를 포함해야 합니다. 같은 프레임의 masked reconstruction만으로 학습한 모델은 조건을 충족하지 않으며, 그런 목적함수를 사전학습에 쓰는 것은 허용합니다. 그 밖의 학습 절차는 자유입니다. 이 조건은 보고서에 선언합니다.

모든 그래프에 요구하는 성질은 2가지입니다. 선언된 입력만의 함수일 것(결정성), 호출 사이에 `ctx` 밖의 상태를 남기지 않을 것(stateless)입니다. 채점기는 임의 호출을 같은 입력으로 재실행해 **공표된 수치 허용 오차 안에서** 같은 출력이 나오는지 확인하고, 난수·시간 의존 연산은 그래프 검사 단계에서 차단합니다. 확률적 요소는 러너의 것입니다 — 각 블록의 시작 잠재값은 러너가 공표된 시드 묶음에서 공급하며, 모델이 자체 난수를 뽑으면 이 검사에 실패합니다. `visual_encode`와 `visual_decode`는 프레임 단위의 stateless 함수입니다.

스타터 킷의 제출 템플릿은 이 인터페이스의 그래프별 함수 형식을 그대로 담고 있으며, 공통 stem 외의 모델 본체는 TODO 블록으로 비워 둡니다.

제출 모델은 ONNX 그래프로만 실행합니다. 임의 Python 코드, 사용자 정의 연산자, 외부 프로세스, 입력별 조회표는 허용하지 않습니다. 텐서 모양, 인덱스, 마스크, 정규화 상수, 정제 스케줄 상수 등 실행에 필요한 구조 상수는 허용합니다. 학습된 값과 평가 사례를 기억한 값은 모두 가중치 묶음에 포함하여 파라미터 수로 집계합니다.

소문제 구성: 하나의 모델, 두 가지 평가

3-1은 개루프(open-loop) 미래 예측으로 모델이 환경의 운동을 학습했는지 평가합니다. 3-2는 페루프(closed-loop) 제어로 같은 표현이 행동 선택에 유효한지 평가합니다.

3-1

개루프 미래 예측

WORLD
MODEL

과거 관측과 미래 액션열만으로 새 관측 없이 미래 영상을 연속 생성합니다.

3-2

페루프 제어

WORLD +
ACTION

매 스텝 실제 화면으로 내부 상태를 갱신하고 같은 상태 표현에서 카트에 가할 힘을 선택해
진자 2개를 세웁니다.

제출물 하나가 두 소문제를 모두 받습니다. 공통 stem 그래프 `visual_encode` · `visual_decode` 와
참가자 그래프 `encode`, `predict_step`, `update`, `observe`, `act` 5개로 이루어지며, 참가자
그래프는 모두 같은 `ctx` 를 공유합니다.

1. 개루프 미래 예측

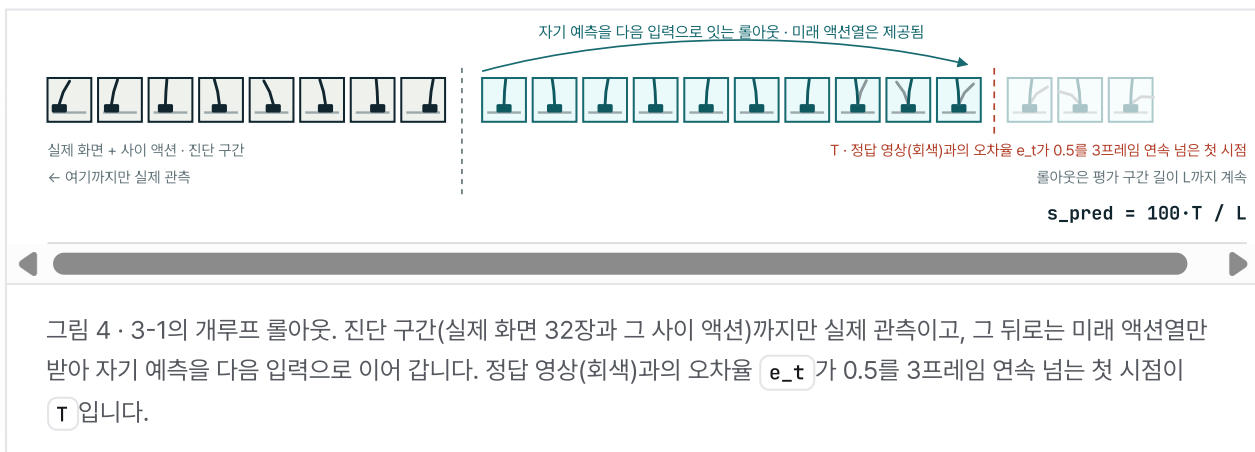
평가 시작 시 모델은 실제 환경의 과거 영상 32개와 그 사이에 실행된 액션을 받습니다. 주최 측은 이 진단 구간의 액션을 카트와 진자의 반응 특성이 드러나도록 구성합니다.

이어서 미래 액션열 전체를 제공합니다. 이후 모델은 실제 환경의 다음 화면을 받지 못하며, 자신이 예측한 잠재값을 출발점으로 미래 영상을 블록 단위로 이어서 생성해야 합니다.

```
context frames : o[0] ... o[31]
context actions : a[0] ... a[30]
future actions : a[31] ... a[31+L-1]
target frames : o[32] ... o[32+L-1] # L = 평가 구간의 예측 프레임 수
```

```
grids = visual_encode(o[0..31]) # 공통 stem · 프레임마다
ctx = encode(grids, a[0..30])

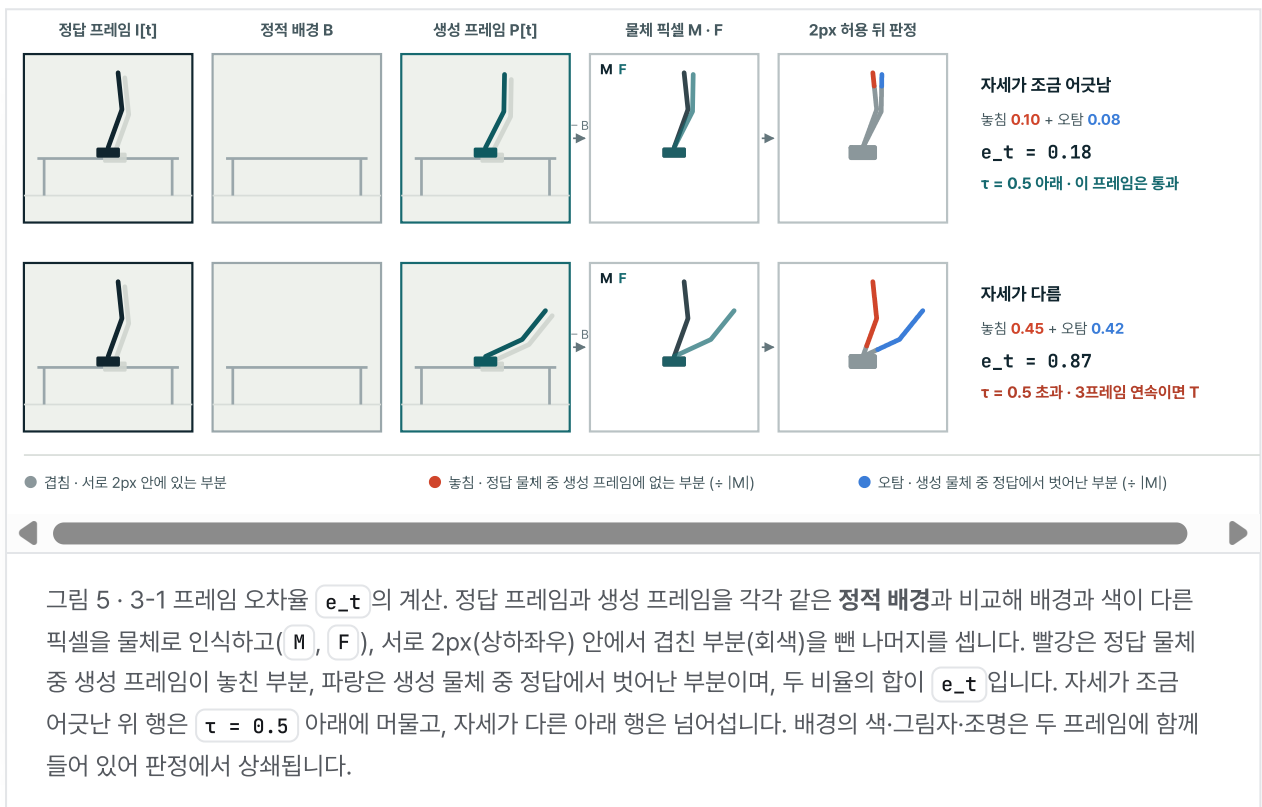
for each block of K frames:
    z = runner.initial_latent(seed) # 공표된 시드 묶음
    for i in range(N):
        z = predict_step(z, level[i], step[i], actions[K], ctx)
    frames[K] = visual_decode(z) # 공통 stem
    ctx = update(ctx, z, actions[K]) # 정답 프레임 없음
```



미래 액션은 사전에 고정되며 예측 도중 정답 프레임으로 내부 상태를 보정할 수 없습니다. 평가 항목은 생성 영상에 그려진 기계가 비공개 정답 영상의 기계와 겹쳐 있는 길이입니다.

정답 영상과의 직접 비교

주최 측은 동일한 비공개 초기 상태와 액션열을 비공개 환경에 적용해 정답 궤적을 만들고, 이를 같은 렌더러로 그린 정답 프레임 $I[t]$ 를 얻습니다. 같은 렌더러로 그 에피소드의 정적 배경 B (방·레일·다리와 그 그림자, 기계는 화면 밖)도 한 장 그립니다. 생성 프레임 $P[t]$ 는 정답 프레임과 직접 비교합니다.



물체 판정. 물체는 배경과 색으로 구분되어야 인식됩니다. 픽셀 p 의 색 차이 $d(p) = I(p) - B(p)$ 에서 세 채널의 평균을 뺀 색상 성분의 크기가 30 그레이 레벨을 넘으면 그 픽셀은 물체입니다. 그림자·색조·조명처럼 세 채널에 고르게 더해지는 변화는 채널 평균과 함께 제거됩니다.

$$\text{obj}(p) = [\|d(p) - \bar{d}(p) \mathbf{1}\|_2 > 30], \quad d(p) = I(p) - B(p), \quad \bar{d}(p) = \frac{1}{3} \sum_c$$

프레임 오차율 e_t 는 놓침 비율과 오탐 비율의 합입니다(그림 5). 정답 물체의 픽셀 중 생성 물체에서 2px 넘게 떨어진 것과, 생성 물체의 픽셀 중 정답 물체에서 2px 넘게 떨어진 것을 각각 정답 물체의 픽셀 수로 나누어 더하고, 1을 넘으면 1로 둡니다. 자세가 맞으면 0, 몇 도 어긋나면 0에 가깝고, 자세가 다르면 1입니다.

$$T = \min \{ t : e_t > 0.5, e_{t+1} > 0.5, e_{t+2} > 0.5 \} \quad (\text{없으면 } L)$$

$$s_{\text{pred}} = 100 \frac{T}{L}, \quad S_{\text{pred}} = \text{mean}_{\text{episode, seed}} (s_{\text{pred}})$$

임계값은 $\tau = 0.5$ 하나입니다. 오차율이 τ 를 3프레임 연속 넘는 첫 시점이 유효 길이 T 이고, 에피소드 점수는 $100 \cdot T / L$ (점)이므로 s_{pred} 는 100점 만점입니다. L 은 평가 구간의 예측 프레임 수로, 모든 에피소드에서 같으며 스타터 킷에 공개합니다. 체크 단위($k > 1$) 생성 모델도 T 는 프레임 단위로 측정하므로 블록 중간에서 종료될 수 있습니다.

에피소드 점수는 공표된 고정 시드 묶음의 시드마다 채점해 평균하며(best-of-N이 아님), s_{pred} 는 에피소드와 시드에 대한 **단순 평균**입니다. 평가 세트는 물리·화면의 모든 축에서, 그리고 운동 구간별로 (매달린 채 약하게 구동되는 구간, 매달린 채 에너지를 높이는 구간, 크게 흔들리는 구간, 흔들어 올려 받아내는 구간, 직립에서 넘어지는 구간) 고르게 뽑습니다. 진자 길이 조합별 성적표와 구간별 평균 점수는 참고용으로 공개합니다.

영상 유효성은 실행 계약으로만 검사합니다. 출력 shape나 프레임 수 오류, NaN/Inf, 허용 RGB 범위 위반, 실행 실패나 제한 시간 초과만 실격 사유입니다. 정지 영상, 과거 프레임 복사, 물체 누락, 임의 표식은 위 오차율로 채점됩니다.

진단 구간

진단 구간은 32프레임(사이 액션 31개)이며 두 가지입니다. 직립 상태에서 시작하지 않는 에피소드는 모든 에피소드에 공통인 같은 액션열을 실행합니다. 직립 상태에서 시작하는 에피소드는 주최 측의 균형 제어기가 기계를 세운 채 카트를 수 cm 왕복시키며, 이 제어기의 액션은 에피소드마다 다릅니다. 어느 쪽이든 실제로 실행된 액션 31개가 `encode`에 전달됩니다. 배포 데이터의 에피소드도 이 두 가지 중 하나로 시작합니다.

2. 페루프 제어

3-2에서 모델은 비공개 환경의 카트를 직접 제어합니다. 목표는 진자 2개를 세워 직립 상태를 최대한 오래 유지하는 것입니다. 평가 에피소드의 시작 상태는 세 가지이며 각각 약 1/3씩입니다: 이미 서 있는 **직립**, 진자가 에너지 일부를 가진 채 흔들리는 **중간 스윙**, 아래로 늘어진 **매달림**.

에피소드 시작 시 주최 측이 준비한 진단용 액션을 먼저 실행합니다. 모델은 이때의 화면 32개와 그 사이의 액션 31개로 현재 운동 상태와 카트·진자의 반응 특성을 파악합니다. 직립 시작에서는 진단 구간 동안 주최 측 균형 제어가 기계를 세워 둡니다(3-1의 진단 구간 참조). 제어 점수는 이 구간이 끝난 뒤부터 계산하며, 그때부터 모델이 매 스텝 다음 액션을 선택합니다 — 직립 시작이라면 그 순간부터 균형을 이어받습니다.

```
action[t] = act(ctx[t])
frame[t+1] = hidden_environment.step(action[t])
grid[t+1] = visual_encode(frame[t+1])           # 공통 stem
ctx[t+1] = observe(ctx[t], grid[t+1], action[t])
```

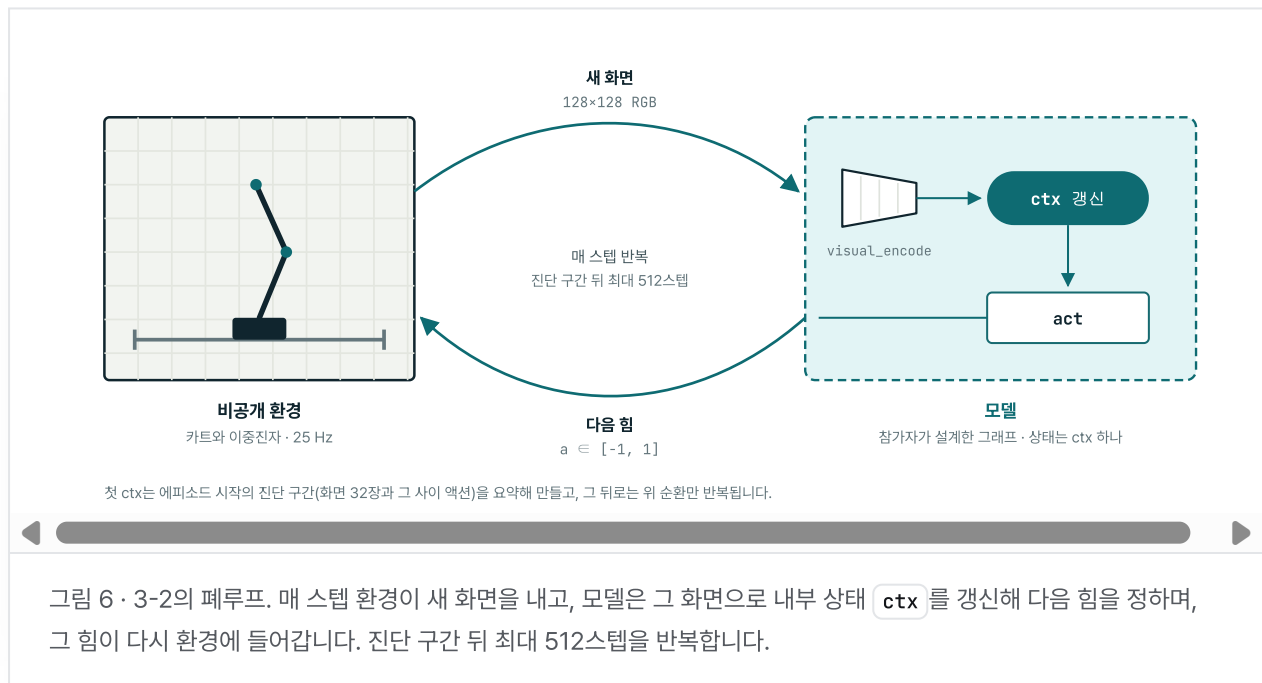


그림 6 · 3-2의 페루프. 매 스텝 환경이 새 화면을 내고, 모델은 그 화면으로 내부 상태 **ctx**를 갱신해 다음 힘을 정하며, 그 힘이 다시 환경에 들어갑니다. 진단 구간 뒤 최대 512스텝을 반복합니다.

액션 실행 후 비공개 환경의 실제 다음 화면을 모델에 전달하고, 모델은 이 화면으로 내부 상태를 갱신한 뒤 다음 액션을 선택합니다. 3-1과 달리 매 스텝 실제 관측으로 예측 오차를 보정할 수 있으며, 이를 페루프 평가라 합니다.

제어 평가는 진단 구간 이후 512스텝 동안 진행하며, 점수는 **두 진자가 서 있었던 시간**입니다. 매 스텝, 두 진자가 모두 수직에서 0.45 rad (약 26°) 이내에 있고 그런 스텝이 **5스텝(0.2초) 이상 연속된 구간**에 속하면 그 스텝을 '서 있음'으로 셉니다.

$$u_t = [|\theta_1| < 0.45 \wedge |\theta_2| < 0.45 \text{ 가 5스텝 이상 연속된 구간 안}]$$

θ_1, θ_2 는 두 진자의 직립 기준 각도(rad)입니다. 수직을 스쳐 지나가는 스윙은 이 범위를 한두 스텝 만에 통과하므로 세지 않고, 받아서 세운 진자는 작은 흔들림이 있어도 범위 안에 머물러 있습니다. 카트 위치와 액션 크기는 점수에 들어가지 않습니다. 레일 양 끝($|x| = 0.4 \text{ m}$)에는 정지 장치가 있고, 에피소드는 항상 512스텝을 채웁니다. 세 가지 시작 상태 각각의 범위는 스타터 킷에 공개합니다. 카메라 시야는 유한합니다: 카트가 레일 끝에 있을 때 두 진자를 바깥으로 수평에 가깝게 뺀으면 진자 끝이 화면 밖으로 최대 약 10 cm 나갈 수 있으며, 그 경우에도 채점은 그대로 계속됩니다.

$$S_{\text{ctrl}} = 100 \frac{1}{512} \sum_t u_t$$

밀기(외란). 평가 에피소드마다 밀기 일정이 미리 한 번 정해져 있으며, 모든 참가자에게 같은 일정이 적용됩니다. 예정된 시점에 기계가 **서 있을 때만**(직전 5스텝 동안 두 진자가 모두 0.45 rad 이내) 실제로 밀고, 서 있지 않으면 그 밀기는 건너뛴니다(미루지 않음). 첫 시점은 진단 구간 뒤 100~140스텝 사이이고 이후 40~100스텝 간격이라 에피소드당 4~8회(평균 약 6회)입니다. 밀기는 카트 또는 두 진자 중 하나의 한 지점(피벗에서 링크 길이의 50~100%)에 가하는 좌우 방향의 수평 힘으로, 1~3스텝 동안 이어집니다. 세기는 약~중 수준으로, 충격량 기준 카트 $0.08 \sim 0.32 \text{ N}\cdot\text{s}$, 첫째 진자 $0.004 \sim 0.013 \text{ N}\cdot\text{s}$, 둘째 진자 $0.0015 \sim 0.0075 \text{ N}\cdot\text{s}$ 입니다(주최 측의 상태 기반 균형 제어가 가장 센 밀기를 절반쯤 버티는 수준). 밀기는 화면에 그려지지 않고 액션 입력에도 나타나지 않으므로, 모델은 이후 화면으로만 그 영향을 알 수 있습니다. 배포 데이터의 일부 에피소드에도 같은 방식의 밀기가 들어 있습니다.

에피소드마다 위 값을 구하고, s_{ctrl} 은 공통 평가 에피소드 묶음에 대한 평균입니다(0~100점). 진자를 세우지 못하는 정책은 카트를 어떻게 흔들든 0점 근처이고, 실제로 도달할 수 있는 상한은 스윙업에 걸리는 스텝 때문에 100보다 조금 낮습니다(주최 측의 상태 기반 전문가 제어가 약 88점 — 시작 상태별로 직립 약 92점, 중간 스윙 약 90점, 매달림 약 83점. 밀기 없이는 약 95점). 모든 참가자는 같은 초기 상태와 조건 묶음으로 평가합니다.

월드 모델과 정책의 공통 상태 표현

ctx 는 하나의 텐서 묶음으로 선언하며 encode , observe , update 만 이 값을 생성·변경할 수 있습니다. act 는 현재 ctx 만 읽는 출력부입니다. ctx 안의 정책 전용 채널은 허용합니다. 평가 중

추가 학습은 허용하지 않습니다.

평가 기준과 실행 한도

최종 점수는 미래 예측의 유효 길이, 제어 성능, 모델 크기를 함께 반영합니다. 제출물은 아래 실행 한도를 모두 통과해야 합니다.

점수. 제출물은 3-1의 예측 점수 S_{pred} 와 3-2의 제어 점수 S_{ctrl} (각 100점 만점)을 받고, 여기에 모델 크기 점수를 더한 최종 점수로 순위를 정합니다. 최종 점수는 100점 만점입니다. 정책이 아직 없는 참가자는 `act` 가 0을 출력하도록 두고 제출할 수 있으며, 그 경우 S_{ctrl} 은 0 근처가 됩니다.

$$S_{\text{final}} = 0.5 S_{\text{pred}} + 0.4 S_{\text{ctrl}} + 10 b_{\text{size}}$$

$$b_{\text{size}} = \text{clip}\left(\frac{\ln 0.5B - \ln P}{\ln 0.5B - \ln 1M}, 0, 1\right)$$

P 는 파라미터 수이며 공통 stem의 가중치도 포함합니다(그 뒀은 스타터 킷에 수치로 공표). 1M 이하면 모델 크기 점수가 10점, 상한인 0.5B이면 0점이고, 그 사이는 로그 눈금으로 줄어듭니다(예: 2M 8.9점, 10M 6.3점, 100M 2.6점).

자격 기준. 제출 모델의 `S_pred` 가 리더보드 세트에서 측정한 마지막 프레임 복사 출력의 `S_pred` 의 1.15배 이상이어야 순위에 오르고 모델 크기 점수를 받습니다. 미달한 제출은 표에 표시만 됩니다.

실행 한도는 모델 크기, 실행 시간, 메모리입니다. 연산량에는 상한을 두지 않으며, 아래 시간 조건 안의 계산은 탐색을 포함해 무엇이든 허용합니다. 실행 시간은 통과 조건일 뿐이라 더 빠르다고 점수가 오르지 않습니다. 시간은 채점 머신에서 배치마다 측정하고, 청크 생성 모델의 프레임당 생성 시간은 블록 생성 시간을 `K` 로 나눈 값입니다.

통과 조건	기준
모델 크기	파라미터 수 0.5B 이하(여러 그래프가 공유하는 텐서는 한 번만 집계)
미래 프레임 생성	프레임당 100 ms 이하. <code>predict_step</code> 호출은 프레임당 최대 4회(= $N \leq 4K$)
에피소드 초기화	<code>encode</code> 는 공표된 채점 런타임에서 1초 이내. 진단 구간 32프레임에 대해 에피소드마다 1회 실행

통과 조건	기준
페루프 제어	한 스텝의 <code>visual_encode</code> · <code>observe</code> · <code>act</code> 가 공표된 채점 런타임에서 p95 40 ms(25 Hz의 한 스텝) 이하
추론 메모리	모든 그래프와 가중치를 포함한 최대 GPU 메모리 6 GB 이하

채점 런타임은 컨테이너 이미지 다이제스트로 공표하며 ONNX Runtime 버전, 실행 프로바이더와 그 옵션, 준비 실행과 반복 횟수를 모두 고정합니다. 각 그래프의 연산량은 검증기가 세어 참고용으로 표시합니다.

수치 정밀도는 참가자가 정합니다. 제출한 ONNX 그래프의 자료형이 그대로 실행되며, 낮은 정밀도의 이점은 시간·메모리 조건의 여유로 나타납니다. 결정성 요건은 정밀도와 무관하게 유지되며, 기록된 호출을 같은 입력으로 재실행했을 때 **공표된 수치 허용 오차 안에서** 같은 출력이 나와야 합니다.

동점 처리

최종 점수 → `S_ctrl` → `S_pred` → 에피소드 `S_ctrl` 의 하위 10퍼센타일 → 파라미터 수(작은 쪽) → 이른 제출 시각 순으로 가립니다.

제출 형식

제출 방법. 제출물은 압축파일 하나(`.zip` , `.tar.gz`)로 만들어 `track3` 칸에 올립니다. 업로드에 쓰는 도구와 방법은 제출 접수가 열릴 때 이 페이지와 이메일로 안내합니다. 제출 도구는 파일을 올린 뒤 같은 칸에 표식 파일 `<이름>.submit` (제출물의 체크섬 포함)을 둡니다. 표식이 있고 그 체크섬이 올라온 파일과 일치하면 그대로 접수하며, 표식 없이 올라온 파일은 보지 않고 영수증도 없습니다. 같은 이름으로는 다시 올릴 수 없으므로 수정 제출은 새 이름으로 올립니다. 접수 결과와 검증 보고서는 영수증으로 돌려드립니다. **마감 시각까지 표식이 도착해 체크섬이 일치한 제출물만 유효합니다.**

제출물 검증과 수정 창구. 그래프 검증기는 대회 1일차에 공개하며 로컬에서 그대로 돌릴 수 있습니다. 접수 시 사용하는 것과 동일한 도구이므로 통과 여부를 미리 확인할 수 있습니다. 마감 전 72시간 동안 접수 창구가 열려 검증 실패를 회신하며, 그 기간에는 수정 제출이 가능합니다. 제출 압축파일은 4 GB 이하여야 하며, 초과하면 접수 시 사유와 함께 반려합니다. **마감 이후에는 수정할 수 없습니다.**

리더보드와 최종 집계. 리더보드는 대회 기간 중 매일 1회 배치로 채점합니다. 각 배치는 컷오프 시점까지 접수되어 검증을 통과한 참가자별 최신 제출 1건(참가자당 하루 1건)을 리더보드 세트에서 채점합니다. 리더보드 세트는 하나이고 공개하지 않습니다. 배치마다 공개하는 것은 리더보드(최종 점수, `S_pred` , `S_ctrl` , 모델 크기 점수, 파라미터 수, 동점 처리에 쓰는 하위 10퍼센타일, 참고용 연산량)와 링크 길이 3×3 구간별 평균 제어 점수이며, 에피소드별 수치는 마감 후에 공개합니다. 같은 그래프를 다시 올리면 다시 채점하지 않고, 재제출이 없으면 이전 결과가 그대로 실립니다. 위의 자격 기준을 넘지 못한 제출과 실행 한도(통과 조건)를 넘은 제출은 표에 표시만 되고 순위에 들지 않으며, 채점 중 실행에 실패한 제출은 사유 유형과 함께 표시되고 다음 배치에서 다시 채점합니다. **최종 순위는 참가자가 접수한 유효 제출 가운데 최종 점수가 가장 높은 제출로 정합니다.**

`graphs/`

공통 stem 그래프 `visual_encode.onnx` ,
`visual_decode.onnx` (구조는 고정, 가중치는
참가자 학습)에 더해, 참가자 그래프
`encode.onnx` , `predict_step.onnx` ,
`update.onnx` , `observe.onnx` , `act.onnx` 5개

`manifest.json`

제출 루트에 두는 매니페스트.

`input_resolution` , `context_length` (`T_ctx`
 ≤ 32), `chunk_size` `K` , `prediction_steps` `N` ,
`visual_grid` , `future_latent_grid` ,
`latent_dtype` , `ctx` 텐서 규격, 참가자 ID, 외부
데이터·사전학습 가중치 사용 여부를 선언합니다.
가중치는 별도 디렉터리 없이 각 ONNX 그래프
안에 담으며, 여러 그래프가 같은 텐서를 공유하면
검증기가 이를 찾아 파라미터 수에 1회만
집계합니다

`train/`

`REPORT.pdf`

제출 가중치를 만든 학습·변환 스크립트. 결과와 함께 공개하는 용도입니다. 행동 조건부 미래 예측 목적함수로 학습하는 과정이 포함되어야 하며, 공통 stem과 과제 특화 시각 모듈은 배포 데이터로 학습합니다

모델 구조, 데이터 활용 방식, 학습·추론 자원, 핵심 실험, 그리고 **학습 시드를 바꿔 재학습했을 때의 점수 분산**을 정리한 4쪽 이내 보고서. 외부 데이터나 사전학습 가중치를 사용했다면 여기에 함께 밝힙니다